Atlassian
Version Control with Git

Course Material
Module 1
Module 2
Module 3
Module 4
Grades
Notes
Discussion Forums
Messages
Course Info

Version Control with Git

by Atlassian

About this Course

The Version Control with Git course provides you with a solid, hands-on foundation for understanding the Git version control system. Git is open source software originally created by Linus Torvalds. Git manages team files for large and small projects. This allows the team to continuously improve its product. It is used by most major technology companies, and is assumed knowledge for many modern programming and IT jobs. It is a core component of DevOps, continuous delivery pipelines and cloud-native computing. You could also use Git to manage the many continuously improving revisions of that book that you are writing. In this course, you will not learn everything there is to know about Git, but you will build a strong conceptual understanding of the technology, and afterward will be able to confidently dig deeper on any topic that interests you. This course assumes no previous knowledge of Git, but if you do have experience with it, you may find this course to be both useful and challenging. This is especially true if you currently "know just enough Git to be dangerous". There are two paths in this course. Both rely heavily on hands-on labs. One path assumes that you have experience using a command line interface, and the other path uses the Sourcetree graphical client. If you are not experienced with a command line, we highly suggest that you go through the Sourcetree path. Eventually, you might want to go through both paths, so that you can decide which tool to use for specific tasks. Besides, repetition is good for learning :) You can watch the videos and take the quizzes from your phone if you want, but the hands-on labs require you to have a Windows or Mac computer. If you use the command line path through the course, you can also use Linux. This course uses Bitbucket (bitbucket.org) as the hosted provider for remote Git repositories. Bitbucket is free for teams of up to 5 people, including private repositories. However, most of the knowledge that you gain in this course applies to Git itself, and you can apply this knowledge to other hosted Git providers (such as GitHub). This course tries to be as concise as possible. It will probably take you about 5-10 hours to go through one of the two paths, but your mileage may vary.

Show less

Taught by:

Steve Byrnes
Instructional Designer

Language	English Subtitles: Arabic, French, Bengali, Uzbek, Ukrainian, Chinese (Simplified), Greek, Italian, Portuguese (Brazil), Vietnamese, Dutch, Korean, German, Pashto, Urdu, Russian, Thai, Indonesian, Swedish, Turkish, Azerbaijani, Spanish, Dari, Hindi, Japanese, Kazakh, Hungarian, Polish
Hardware Req	Requires a Mac, Windows or Linux operating system to do the hands-on labs.
How To Pass	Pass all graded assignments to complete the course.
User Ratings	★ 4.7

Syllabus

Module 1

Our First Repository

This module gets you started using Git. It provides the foundation for the rest of the course. You will install Git and/or Sourcetree, create a local repository, create a commit, create a remote repository and push commits to a remote repository. You have the option of doing the hands-on labs using the Sourcetree graphical Git client and/or using a command line.

12 videos, 10 readings

1. Reading: Welcome
2. Video: DevOps and Git in a Nutshell
3. Video: Git Overview
4. Video: (Sourcetree) Installation and Getting Started
5. Reading: (Lab- Sourcetree) Installation and Getting Started
6. Video: (Command Line) Installation and Getting Started
7. Reading: (Lab- Command Line) Installation and Getting Started
8. Video: Git Locations
9. Video: (Sourcetree) Create a Local Repository
10. Reading: (Lab- Sourcetree) Create a Local Repository
11. Video: (Command Line) Create a Local Repository
12. Reading: (Lab- Command Line) Create a Local Repository
13. Video: (Sourcetree) Commit to a Local Repository
14. Reading: (Lab- Sourcetree) Commit to a Local Repository
15. Video: (Command Line) Commit to a Local Repository
16. Reading: (Lab- Command Line) Commit to a Local Repository
17. Video: Create a Remote Repository
18. Reading: (Lab) - Create a Remote Repository
19. Video: (Sourcetree) Push to a Remote Repository
20. Reading: (Lab- Sourcetree) Push to a Remote Repository
21. Video: (Command Line) Push to a Remote Repository
22. Reading: (Lab- Command Line) Push to a Remote Repository

Graded: Module 1 Assessment

View Less

Module 2

Branching and Merging I

This module discusses to of the key concepts involved with Git: branching and merging. It starts with foundational knowledge on Git's commit graph model, Git IDs and references. Branching and merging are then covered, including the difference between fast-forward merges and using merge commits.

9 videos, 6 readings

1. Video: Git's Graph Model
2. Video: (Sourcetree) Git IDs
3. Video: (Command Line) Git IDs
4. Video: (Sourcetree) Git References
5. Reading: (Lab- Sourcetree) Git References
6. Video: (Command Line) Git References
7. Reading: (Lab- Command Line) Git References
8. Video: (Sourcetree) Branches
9. Reading: (Lab- Sourcetree) Branches
10. Video: (Command Line) Branches
11. Reading: (Lab- Command Line) Branches
12. Video: (Sourcetree) Merging
13. Reading: (Lab- Sourcetree) Merging
14. Video: (Command Line) Merging
15. Reading: (Lab- Command Line) Merging

Graded: Module 2 Assessment

Module 3

Branching and Merging II

This module continues the discussion of branching and merging. It starts with resolving a merge conflict. Then synchronizing with remote repositories is covered, including the fetch, pull and push commands. The other major topic of this module is rewriting history. The commit history is an important part of a project. A clean commit history makes it easier to understand the project, and then contribute to it by adding new features or fixing problems. If you have not shared your commits, you can rewrite them in a way that makes a cleaner project history. Rewriting history includes amending commits and rebasing.

10 videos, 10 readings

1. Video: (Sourcetree) Resolving Merge Conflicts
2. Reading: (Lab- Sourcetree) Resolving Merge Conflicts
3. Video: (Command Line) Resolving Merge Conflicts
4. Reading: (Lab- Command Line) Resolving Merge Conflicts
5. Video: (Sourcetree) Tracking Branches
6. Reading: (Lab- Sourcetree) Tracking Branches
7. Video: (Command Line) Tracking Branches
8. Reading: (Lab- Command Line) Tracking Branches
9. Video: (Sourcetree) Fetch, Pull and Push
10. Reading: (Lab- Sourcetree) Fetch, Pull and Push
11. Video: (Command Line) Fetch, Pull and Push
12. Reading: (Lab- Command Line) Fetch, Pull and Push
13. Video: (Sourcetree) Rebasing
14. Reading: (Lab- Sourcetree) Rebasing
15. Video: (Command Line) Rebasing
16. Reading: (Lab- Command Line) Rebasing
17. Video: (Sourcetree) Rewriting History
18. Reading: (Lab- Sourcetree) Rewriting History
19. Video: (Command Line) Rewriting History
20. Reading: (Lab- Command Line) Rewriting History

Graded: Module 3 Assessment

Module 4

Git Workflows

This module covers topics on how teams work with Git. Git is flexible enough to work with many types of projects and teams. The better you understand Git's capabilities, the more effectively to can use it in your projects. Topics include pull requests, centralized workflows, feature branch workflows, forking workflows and Gitflow workflows. This module concludes with a project in which you use Git with a Gitflow workflow.

4 videos, 5 readings

1. Video: (Sourcetree) Pull Requests I
2. Reading: (Lab- Sourcetree) Pull Requests I
3. Video: (Command Line) Pull Requests I
4. Reading: (Lab- Command Line) Pull Requests I
5. Video: Pull Requests II
6. Reading: (Lab- Sourcetree) Pull Requests II
7. Reading: (Lab- Command Line) Pull Requests II
8. Video: Git Workflows
9. Peer Review: Version Control with Git Final Project
10. Reading: Final Project

Graded: Module 4 Assessment

How It Works

General

What do start dates and end dates mean?
Once you enroll, you'll have access to all videos, readings, quizzes, and programming assignments (if applicable). If you choose to explore the content without purchasing, you may not be able to access certain assignments. If you don't finish all graded assignments before the end of the session, you can reset your deadlines. Your progress will be saved and you'll be able to pick up where you left off.

What are due dates? Is there a penalty for submitting my work after a due date?
Within a course, there are suggested due dates to help you manage your schedule and keep work from piling up. Quizzes and programming assignments can be submitted late without consequence. However, it is possible that you won't receive a grade if you submit your peer-graded assignment too late because classmates usually review assignment within three days of the assignment deadline.

Can I re-attempt an assignment?
Yes. If you want to improve your grade, you can always try again. If you're re-attempting a peer-graded assignment, re-submit your work as soon as you can to make sure there's enough time for your classmates to review your work. In some cases you may need to wait before re-submitting a programming assignment or quiz. We encourage you to review learning material during this delay.

Show less

Peer-graded assignments

Peer-graded assignments require you and your classmates to grade each other's work.

How do peer graded assignments work?
After you submit your assignment, you will review some of your peers' assignments. The number of assignments you must review is set by the instructor of the course.

I reviewed my peers' assignments! What happens next?
While you're reviewing your peers' assignments, they'll review yours. If you submit your assignment on time, you'll get your grade within a week, as long as at least one peer reviews your assignment. If you submit late, you'll need all of the peer reviews the instructor requires. Learn more about Peer Graded Assignments.

How are grades calculated?
You and your classmates will be asked to provide a score for each part of the assignment. Final grades are calculated by combining the median scores you received for each section.

What kind of feedback should I give?
Use the instructor's criteria in the rubric to grade honestly and fairly. If your peers' answers are excellent, score them highly and tell them what they did well. If their answers aren't as good, give the score they deserve, and be sure to provide respectful, useful feedback so they can do better next time they attempt the assignment.

Is there a penalty for submitting my work late?
No, but it's important to submit your work as close to the due date as you can. Classmates grade most of the assignments within three days of the due date. If you submit yours too late, there may not be anyone to review your work.

If I fail an assignment, can I try again?
Yes! You can always try again, but you'll need to resubmit your work as soon as possible to make sure your classmates have enough time to grade your work.

Can I edit my assignment?
Yes, but you'll need to re-submit your work and any grade you've already received will be deleted.

Show less

View the course in catalog